

PROGRAMMING IN JAVA

PROJECT REPORT

Practical and Skills Development

CERTIFICATE

THE TASKS ENTERED IN THIS REPORT HAVE BEEN SATISFACTORILY
PERFORMED BY

Registration No : 25BAI11414
Name of Student : ANUSHKA SHARMA
Course Name : PROGRAMMING IN JAVA
Course Code : CSE2006
School Name : SCAI
Slot : B11+B12
Class ID : BL2026270100415
Semester : FALL 2026/27

Course Faculty Name : Dr. DHERESH SONI

INDEX

<u>S.NO</u>	<u>TITLE</u>	<u>PAGE NO.</u>
1.	INTRODUCTION	3
2.	OBJECTIVES	4
3.	PROBLEM STATEMENT	5
4.	FUNCTIONAL REQUIREMENTS	6
5.	NON-FUNCTIONAL REQUIREMENTS	10
6.	SYSTEM ARCHITECTURE	14
7.	DESIGN DIAGRAMS	19
8.	DESIGN DECISIONS & RATIONALS	29
9.	IMPLEMENTATION	32
10.	SCREENSHOTS AND RESULTS	41
11.	TESTING	53
12.	CHALLENGES FACED	62
13.	LEARNING & KEY TAKEAWAYS	64
14.	FUTURE ENHANCEMENTS	67
15.	CONCLUSION	72
16.	REFERENCES	73

INTRODUCTION

Food plays a vital role in students' welfare, particularly in colleges, hostels and other types of educational establishments where meals have to be arranged for a group of students. If there are a large number of students, it becomes difficult to plan the meals by hand. The individual in charge of the planning must take into account the number of students, the budget available, the ingredients on hand and whether the chosen meal is appropriate for the students.

“FoodWise” is a system based on Java which has been developed in order to simplify the process of meal planning and the optimisation of resources. It enables an administrator to keep records of information about students, ingredients and meals and then produces a suitable meal recommendation according to the number of students and the budget available.

The project uses Java as the main programming language and MySQL for storing the application data. JDBC is used to connect the Java application with the database. The system also applies different Java concepts such as object-oriented programming, inheritance, abstraction, interfaces, collections, exception handling and file handling.

The cheapest meal is not the only factor that the FoodWise recommendation system takes into account: it also considers factors such as budget usage, the suitability of the meal and resource efficiency. Before it makes a recommendation, the system verifies that there is sufficient quantities of the required ingredients for the number of students who have been selected.

The project was designed as a practical application of the concepts acquired in Java and centers on solving a small but realistic student welfare problem, at the same time keeping the system simple so that it can be understood, maintained and extended in the future.

PROBLEM STATEMENT

Planning meals for a group of students requires making a number of decisions at one time. A meal might be cheap but the ingredients needed could be unavailable in sufficient amounts. On the other hand, a meal that is more suitable might use a larger part of the budget. When these factors have to be checked by hand each time, the process can become time-consuming and errors are also likely to occur.

In most cases when planning simple meals there is no single location at which information about the students, the stock of ingredients, the details of the meals and earlier meal plans can all be managed. As a result it becomes more difficult to keep track of the resources available and of previous decisions.

What **FoodWise** is aiming to achieve is the creation of a system based on Java which is able to store and manage this information and then give a meal recommendation in accordance with the specified requirements.

The system takes inputs such as:

- ✓ Number of students
- ✓ Available budget
- ✓ Available ingredients
- ✓ Meal cost
- ✓ Meal suitability
- ✓ Ingredient quantity required for a meal

Using these inputs, FoodWise looks at the meals that are available and then suggests one that meets both the budget and ingredient availability requirements as well as taking into account its suitability and resource efficiency. The plan produced includes the total cost, the amount of the remaining budget, the recommendation score and other pertinent information, and it can also be stored in the database for future reference.

OBJECTIVES

The main objectives of the FoodWise project are:

1. To develop a Java-based system for managing student meal planning.
2. To maintain student, ingredient and meal information in a structured manner.
3. To check whether sufficient ingredients are available for the required number of students.
4. To recommend a suitable meal based on budget, suitability and resource efficiency.
5. To calculate the total meal cost and remaining budget automatically.
6. To save generated meal plans using a MySQL database.
7. To provide a simple way to view previously saved meal plans.
8. To apply Java programming concepts such as **OOP, inheritance, abstraction, interfaces, collections, exception handling and JDBC** in a practical project.
9. To generate a basic report containing the details of the recommended meal plan.

FUNCTIONAL REQUIREMENTS

FoodWise consists of modules that perform specific functions within the overall system. The modules connect with each other, from the management of the basic data to the creation and storage of a meal plan.

The system's primary functional requirements are listed below.

4.1 Student & Welfare Management

This module is used to update the basic information of the students which are taken in consideration while planning meals.

Main functions:

- i. Add a new student.
- ii. See the list of students.
- iii. Update student info.
- iv. Remove a student record.
- v. Store the student name, age and group/category information.
- vi. Input:
- vii. Student name
- viii. Age
- ix. Group name
- x. Output:
- xi. Successfully added/updated/deleted student message.
- xii. List of students stored in the database.

This module utilizes CRUD operations and stores the information in the **students** table.

4.2 Ingredient Management

This module manages the ingredients which are available for preparing meals.

Main functions:

- Add new ingredients.
- View available ingredients.
- Update the available quantity of an ingredient.
- Store the unit and cost of each ingredient.
- Check ingredient availability during meal recommendation.

Input:

- Ingredient name
- Available quantity
- Unit
- Cost per unit

Output:

- Updated ingredient information.
- List of available ingredients.
- Availability status while generating a recommendation.

The ingredient information is stored in the **ingredients** table. The system checks the quantity required by a meal against the available quantity before considering that meal for recommendation.

4.3 Meal Management

This module is used to maintain the meals which can be recommended by FoodWise.

Main functions:

- Add a new meal.
- View available meals.
- Store meal type.
- Store cost per student.
- Store suitability score.
- Maintain the ingredients required for each meal.

Input:

- Meal name
- Meal type
- Cost per student
- Suitability score
- Required ingredients

Output:

- List of available meals.
- Meal information used by the recommendation system.

The meal details are stored in the **meals** table. The relationship between meals and their ingredients is maintained through the **meal_ingredients** table.

4.4 Intelligent Meal Recommendation

This is the main decision-support module of FoodWise..

The user provides:

- Number of students
- Available budget

The system then checks the available meals. A meal is considered only when the required ingredients are available in sufficient quantity for the selected number of students.

After this, the recommendation system calculates a score using three factors:

- **Budget score – 30%**
- **Suitability score – 40%**
- **Resource efficiency score – 30%**

The meal with the highest calculated score is selected as the recommendation.

Output:

- Recommended meal
- Meal type
- Number of students
- Cost per student
- Total cost
- Available budget
- Remaining budget
- Suitability score
- Recommendation score
- Plan date

For example, during testing, FoodWise recommended **Vegetable Khichdi** for 5 students with a budget of Rs.800. The total cost was Rs.140 and the remaining budget was Rs.660.

4.5 Meal Plan and Reporting

After a recommendation is generated, the administrator can save the meal plan.

Main functions:

- Save a generated meal plan.
- View previously saved meal plans.
- Store budget and cost details.
- Store recommendation score.
- Store the date of the plan.
- Generate a text report for the generated plan.

Output:

The saved plan contains:

- Plan ID
- Meal ID
- Number of students
- Budget
- Total cost
- Remaining budget
- Recommendation score
- Plan date

This information is stored in the **meal_plans** table and can be retrieved later from the application.

4.6 Overall Input and Output Flow

The basic functional flow of FoodWise can be represented as:

Student / Ingredient / Meal Data → User enters students and budget → Ingredient availability check → Meal filtering → Recommendation score calculation → Best matching meal → Cost calculation → Save Meal Plan → View / Generate Report

This gives the application a clear workflow instead of treating each module as a separate feature.

Functional Requirements Summary

Module	Main Purpose
Student Management	Manage student information
Ingredient Management	Manage ingredient stock and cost
Meal Management	Manage meals and their details
Meal Recommendation	Select a suitable meal using scoring
Meal Plan & Reporting	Save, retrieve and report generated plans

NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define how the FoodWise system should behave, rather than what specific features it should provide. These requirements were considered while building the project in order to ensure that the application remains simple to use, reliable and easier to maintain.

The main non-functional requirements of FoodWise are listed below.

5.1 Usability

The system should be easy for an administrator to understand and operate.

FoodWise features a menu-driven console interface, where the available options are displayed clearly. The user can select different modules using numbers and enter the required information.

The application also displays messages such as:

- Successful database connection
- Login successful or failed
- Student added successfully
- Meal plan saved successfully
- Recommendation result
- Validation and error messages

This makes the system easier to use, even without a complicated graphical interface.

5.2 Reliability

The system should give consistent results when valid information is entered.

FoodWise uses the MySQL database to store student, ingredient, meal and meal-plan information. The saved information can be retrieved later, instead of being lost when the application is closed.

The recommendation process also checks ingredient availability before selecting a meal, which reduces the chance of recommending a meal which cannot actually be prepared due to insufficient ingredients.

5.3 Error Handling

The application should handle incorrect input and database-related problems without suddenly terminating the program.

FoodWise uses exception handling, particularly for database operations using SQLException.

Input validation is also performed for important values. For example:

- Student age is validated.
- Student name cannot be empty.
- Ingredient quantity cannot be negative.
- Meal cost must be positive.
- Suitability score must be between 0 and 100.
- Number of students must be greater than zero.
- Budget must be greater than zero.

When an error occurs, the application displays a message to the user, rather than exposing the complete technical error or terminating unexpected.

5.4 Security

The application includes a basic administrator login before the main system menu can be accessed.

The username and password are checked against the users table in the database. Database queries which receive user input use PreparedStatement, which also helps to avoid placing user input directly inside SQL queries.

The project is designed as a local academic application, so the security level is appropriate for the current scope. It is not intended to be a production-level authentication system.

5.5 Maintainability

The project is split into separate packages and classes, rather than having all the code in one place.

The main packages include:

foodwise

├─ model

├─ dao

├─ service

├─ exception

├─ util

└─ main

For example, database operations are handled by DAO classes, and recommendation-related logic is handled by service classes.

This separation makes it easier to locate and modify a particular part of the application, without having to change the entire system.

5.6 Performance

The FoodWise application is designed to address the needs of small- and medium-scale student groups.

As such, the use of database-heavy operations such as SQL queries and PreparedStatement is acceptable, as the system does not process the entire database in one operation. The recommendation algorithm only takes meals requiring ingredients available in the selected amount. The choice is determined by calculating a matching index only for applicable meals.

Given the console-based nature of the application and its primarily educational purpose, performance is not a priority.

5.7 Resource Efficiency

The question of resource efficiency is two-sided in this system.

First, the application takes user input on what quantities of ingredients they possess. It then uses this information to determine a meal plan that does not feature recipes requiring more ingredients than the user has. Additionally, the recommendation algorithm incorporates a resource-efficiency score based on the cost per portion.

As such, the FoodWise system does not recommend meals based solely on their general suitability and recipe matching index but also considers the availability of resources.

5.8 Scalability

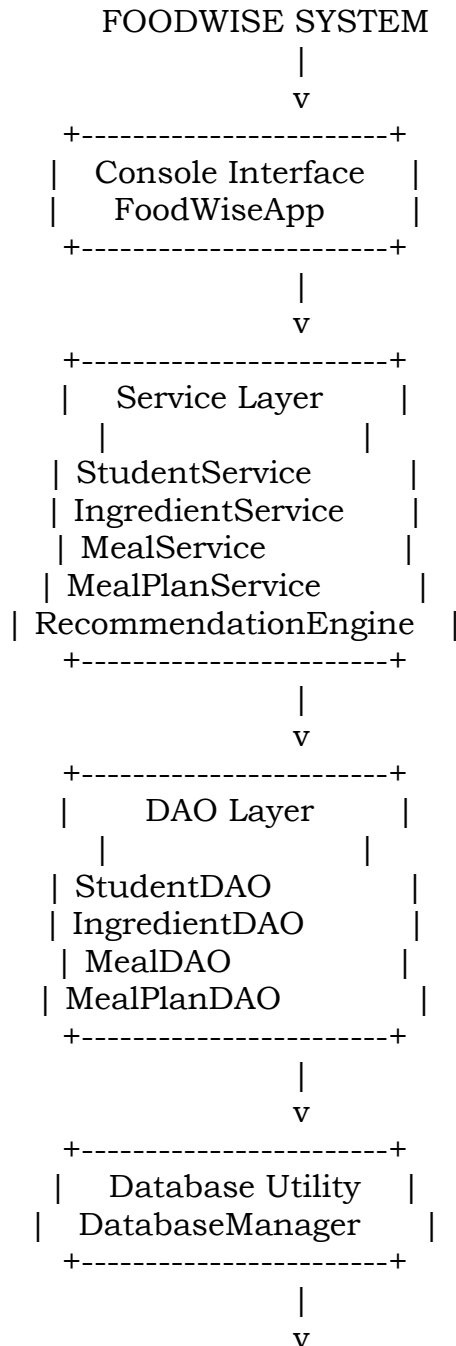
The FoodWise application is not designed to handle large bodies of data for now. Its primary intended use is a single student group, and the application's codebase is written in a modular fashion to allow the addition of new features.

Therefore, the application can scale to larger portions of data in the future, which will be necessary if it is to be adapted to real-life use. New categories, ingredients, users, and other elements can be added to the system without rewriting the source code.

SYSTEM ARCHITECTURE

The FoodWise system follows a layered architecture. The main purpose of using this structure is to keep the user interface, application logic and database operations separated from each other.

The application is developed as a Java console-based system. When the administrator enters information, it is passed through the appropriate service class. The overall architecture can be represented as:





6.1 Presentation Layer

The main functionality is provided by the presentation layer (FoodWiseApp.java) through the menu-driven interface that allows the administrator to:

- Log in
- Manage students
- Manage ingredients
- Manage meals
- Generate recommendation
- View saved meal plans
- Generate reports
- Exit

The application uses input from the console and displays appropriate information back to the user. As observed in the code, the menu and console messages are hard-coded for simplicity, but can be extracted into a property file in a more production-ready system.

6.2 Service Layer

The service layer contains the core processing and validation logic for the application.

The key service classes include:

- StudentService – student information and validation
- IngredientService – ingredient management and quantity validation
- MealService – meal management and suitability/validation
- MealPlanService – recommendation and meal-plan processing
- RecommendationEngine – recommendation score calculation, meal selection

This layer abstracts the business processing from the main application class.

6.3 DAO Layer

DAO (Data Access Object) layer contains the implementation for database access. The DAO interfaces and classes use JDBC API for connecting to the MySQL database.

The FoodWise application has the following DAO classes:

StudentDAO
IngredientDAO
MealDAO
MealPlanDAO

These classes implement database operations using SQL statements with JDBC PreparedStatement, Statement, ResultSet and connection objects. For example, by using the StudentDAO class, one can conveniently perform database operations such as: insert, view, update and delete student records.

6.4 Database Layer

MySQL is used as the database management system for this application. The application's database schema contains the following tables:

users
students
ingredients
meals
meal_ingredients
meal_plans

The meal_ingredients table serves as a “junction” table that establishes a many-to-many relationship between meals and ingredients. This is required because one meal can contain many ingredients and one ingredient can be part of many meals.

The meal_plans table stores the generated recommendation including additional information such as total cost, budget, score, and date.

6.5 Database Connectivity

The DatabaseManager class is used to establish a JDBC connection to the MySQL database. The application's overall architecture can be summarized in the following diagram:



↓
MySQL

6.6 Recommendation Processing

The recommendation generation process involves an additional step since it requires actual decision-making. The general steps are outlined below:

User provides: Students + Budget

↓

Load applicable meals

↓

Check available ingredients

↓

Exclude meals with insufficient ingredients

↓

Check budget constraints

↓

Calculate recommendation score

↓

Compare available meals

↓

Select top recommendation

↓

Calculate totals and update budget

↓

Display recommendation

↓

Save meal-plan (optional)

Currently, the recommendation score is calculated using the following weights:

Budget Score → 30%

Suitability Score → 40%

Resource Efficiency → 30%

This process extends the basic CRUD operations by adding a decision-making component for selecting a suitable meal.

6.7 Architecture Choice

A layered (n-tier) architecture was selected for this project because such design is easier to maintain and modify. For example, if one wanted to change the underlying database or switch to another database management system, such change would require minimal code adjustments. This is because all database-related operations are encapsulated within the DAO layer. In this project, the DAO layer also serves as the separation between business logic and persistence.

I also wanted to demonstrate practical application of various Java concepts and design patterns. Therefore, it was important to separate concerns and organize code logically. In this project, the code is separated into model, DAO, service, utility, and exception packages, as opposed to putting everything in a single class or package.

DESIGN DIAGRAMS

7.1 USE CASE DIAGRAM

For the FoodWise Use Case Diagram, keep it **simple**. We have one main actor:
Administrator.

Actor

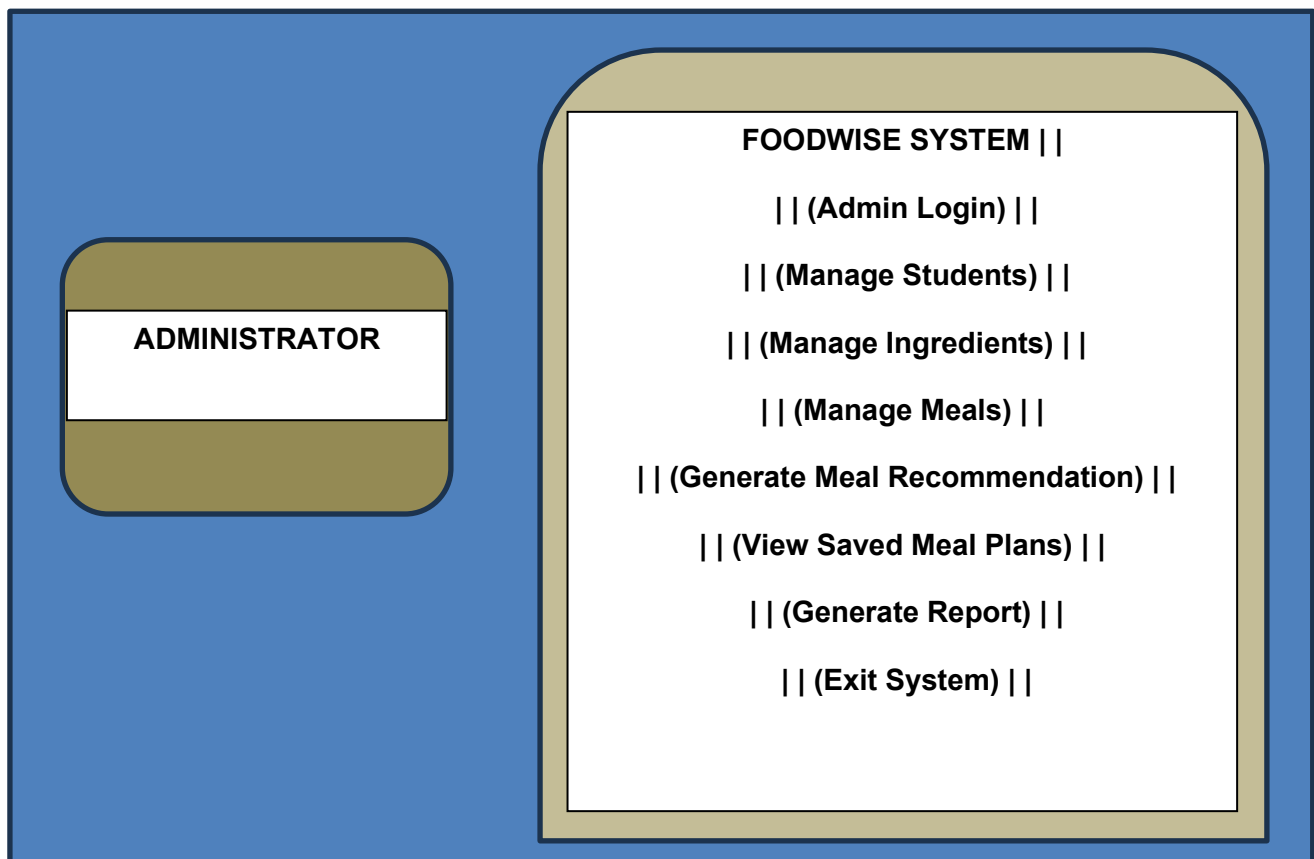
Administrator

The administrator interacts with FoodWise to manage the data and generate meal recommendations.

Use Cases

Connect the **Administrator** to these use cases:

- + Admin Login
- + Manage Students
- + Manage Ingredients
- + Manage Meals
- + Generate Meal Recommendation
- + View Saved Meal Plans
- + Generate Report
- + Exit System



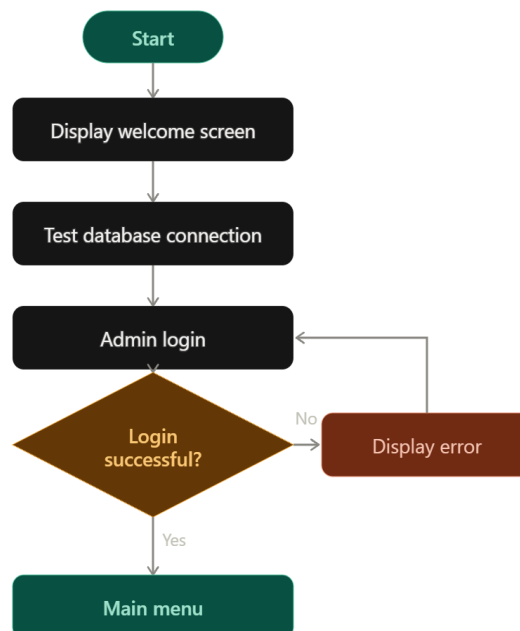
The Use Case Diagram represents the interaction between the administrator and the FoodWise system. The administrator logs into the application and can perform student, ingredient and meal management operations. The administrator can also generate a meal recommendation by providing the required number of students and budget. Generated plans can be saved and viewed later, and a report can be generated from the application.

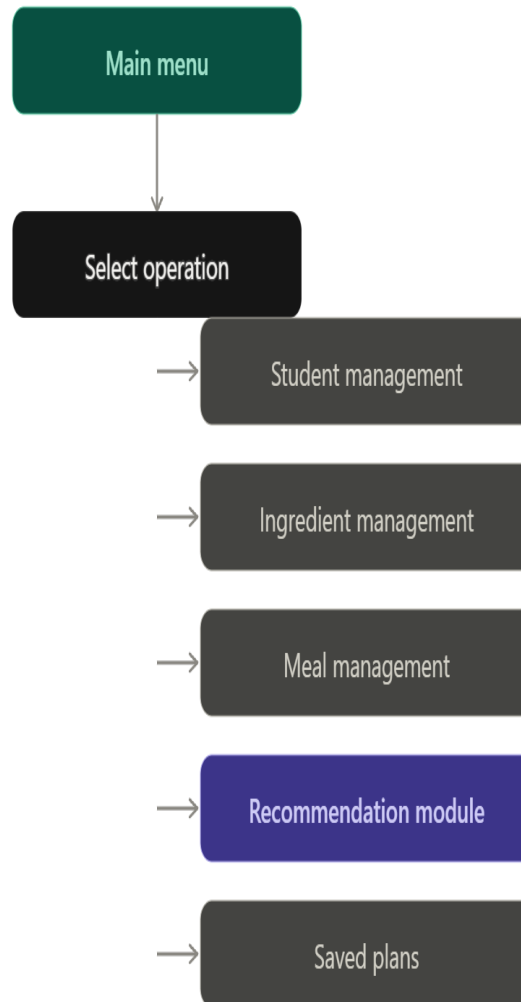
7.2 WORKFLOW DIAGRAM

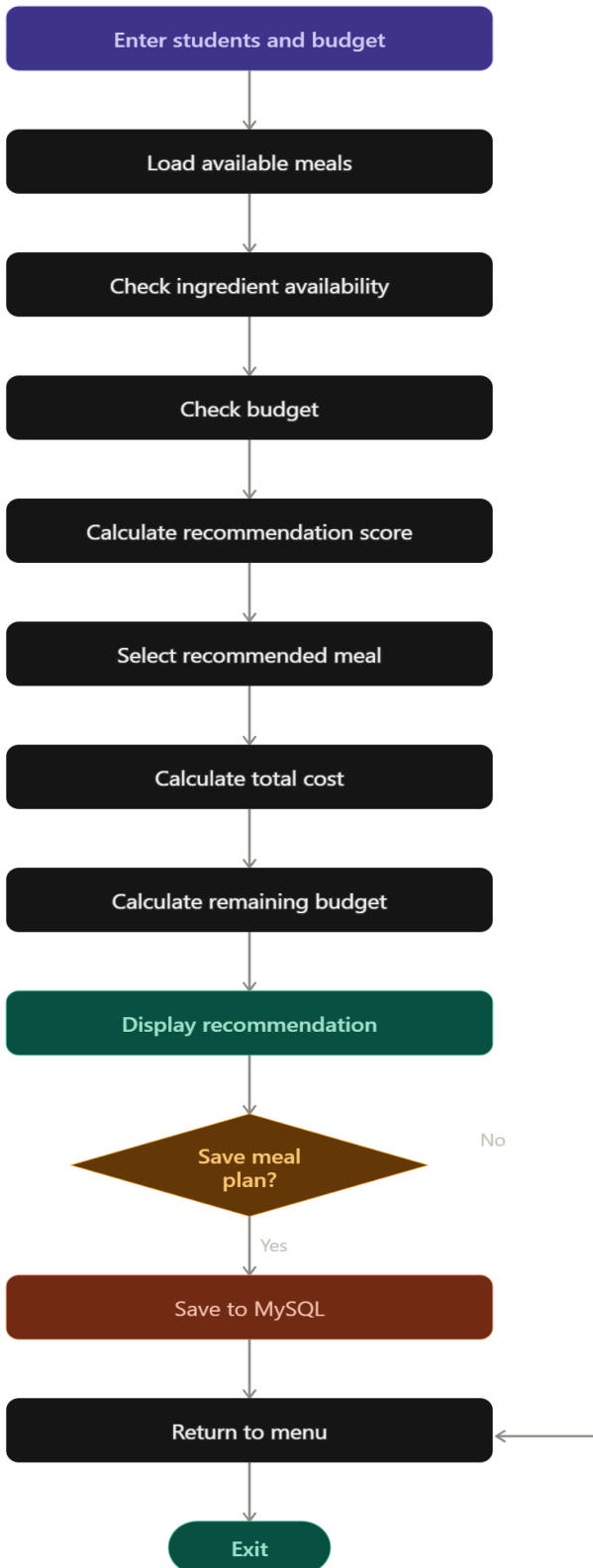
The Workflow Diagram represents the overall sequence followed while using FoodWise.

The workflow starts when the application is opened and the database connection is tested. The administrator then logs in and accesses the main menu. Depending on the selected option, the administrator can manage the stored information or generate a recommendation.

For recommendation generation, the system receives the number of students and available budget. It checks ingredient availability, filters the meals and calculates the recommendation score. The selected meal's total cost and remaining budget are then displayed. The administrator can optionally save the generated meal plan in the MySQL database.







7.3 SEQUENCE DIAGRAM

The Sequence Diagram shows the interaction between the major components when the administrator generates a meal recommendation.

Participants

Administrator

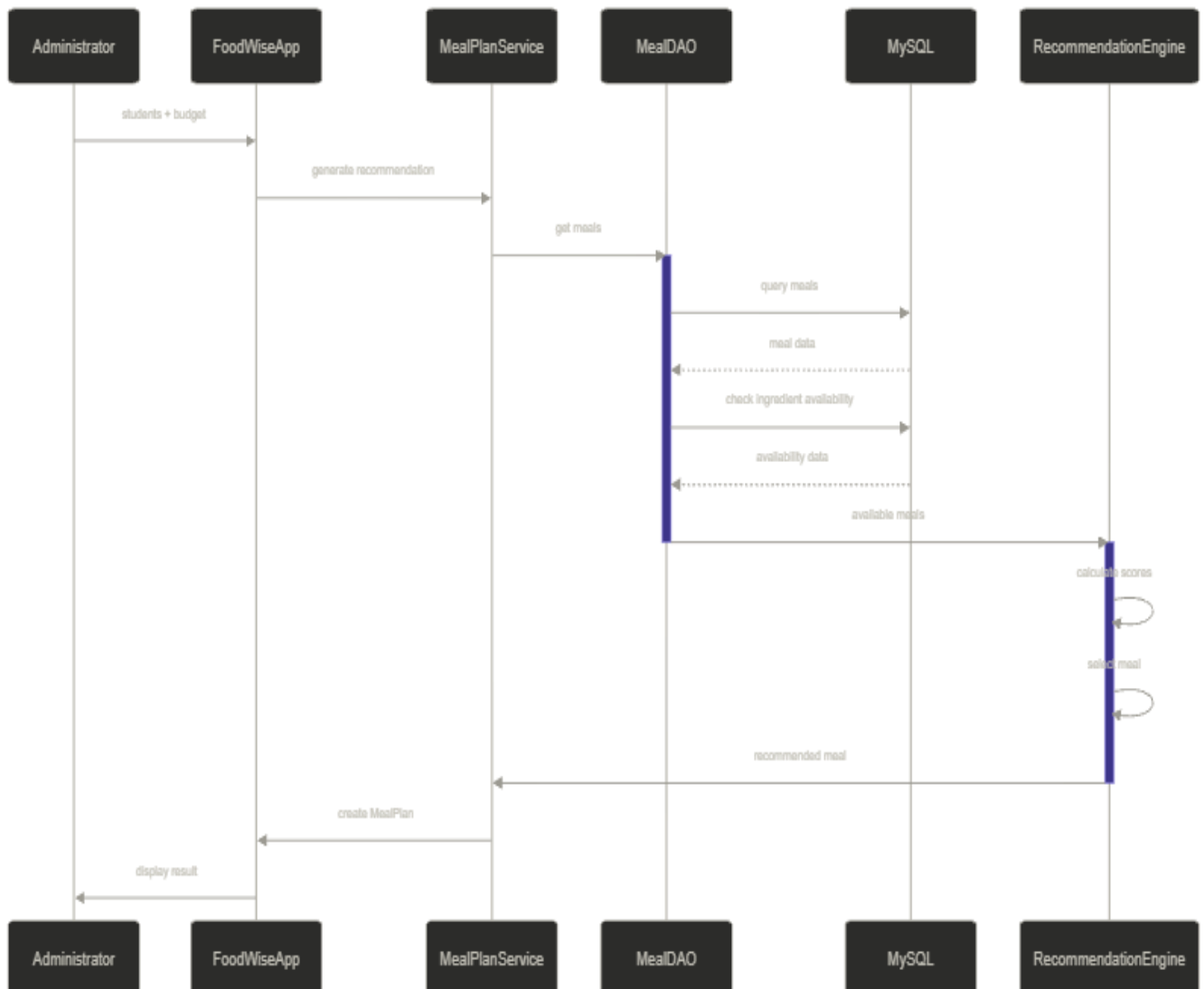
FoodWiseApp

MealPlanService

MealDAO

RecommendationEngine

MySQL Database

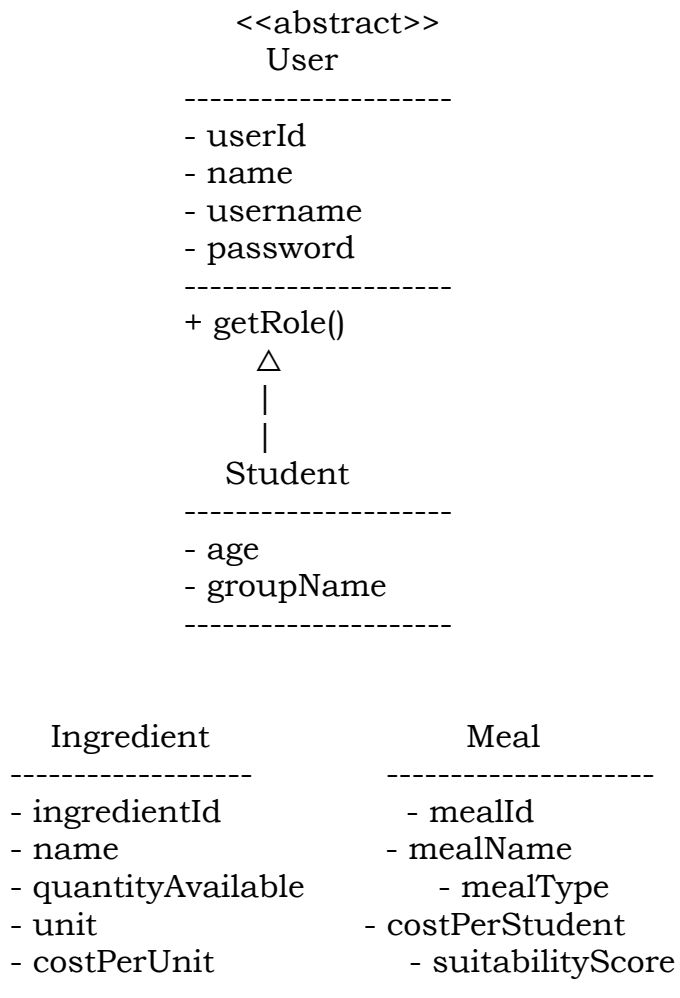


Description

The sequence begins when the administrator enters the number of students and available budget. FoodWiseApp passes these values to MealPlanService. The service retrieves available meals through MealDAO and checks the ingredient availability. The eligible meals are then passed to RecommendationEngine, which calculates the recommendation score and selects the meal. The result is returned to the application and displayed to the administrator.

7.4 CLASS DIAGRAM

The Class Diagram represents the main classes used in the FoodWise application and their relationships.



MealPlan

- planId
- mealId
- studentCount
- budget
- totalCost
- remainingBudget
- recommendationScore
- planDate

<<interface>>
RecommendationStrategy

+ recommendMeal()
 △
 |
 |
RecommendationEngine

+ recommendMeal()

7.5 ER DIAGRAM

The Entity Relationship Diagram represents the structure of the FoodWise MySQL database.

Entities

The database contains the following main tables:

USERS

PK user_id
name
username
password
role

STUDENTS

PK student_id
name
age
group_name

INGREDIENTS

PK ingredient_id
name
quantity_available
unit
cost_per_unit

MEALS

PK meal_id
meal_name
meal_type
cost_per_student
suitability_score

MEAL_INGREDIENTS

PK,FK meal_id
PK,FK ingredient_id
 quantity_required

MEAL_PLANS

PK plan_id
FK meal_id
 student_count
 budget
 total_cost
 remaining_budget
 recommendation_score
 plan_date

Relationships

The main relationships are:

MEALS 1 ----- M MEAL_INGREDIENTS M ----- 1 INGREDIENTS

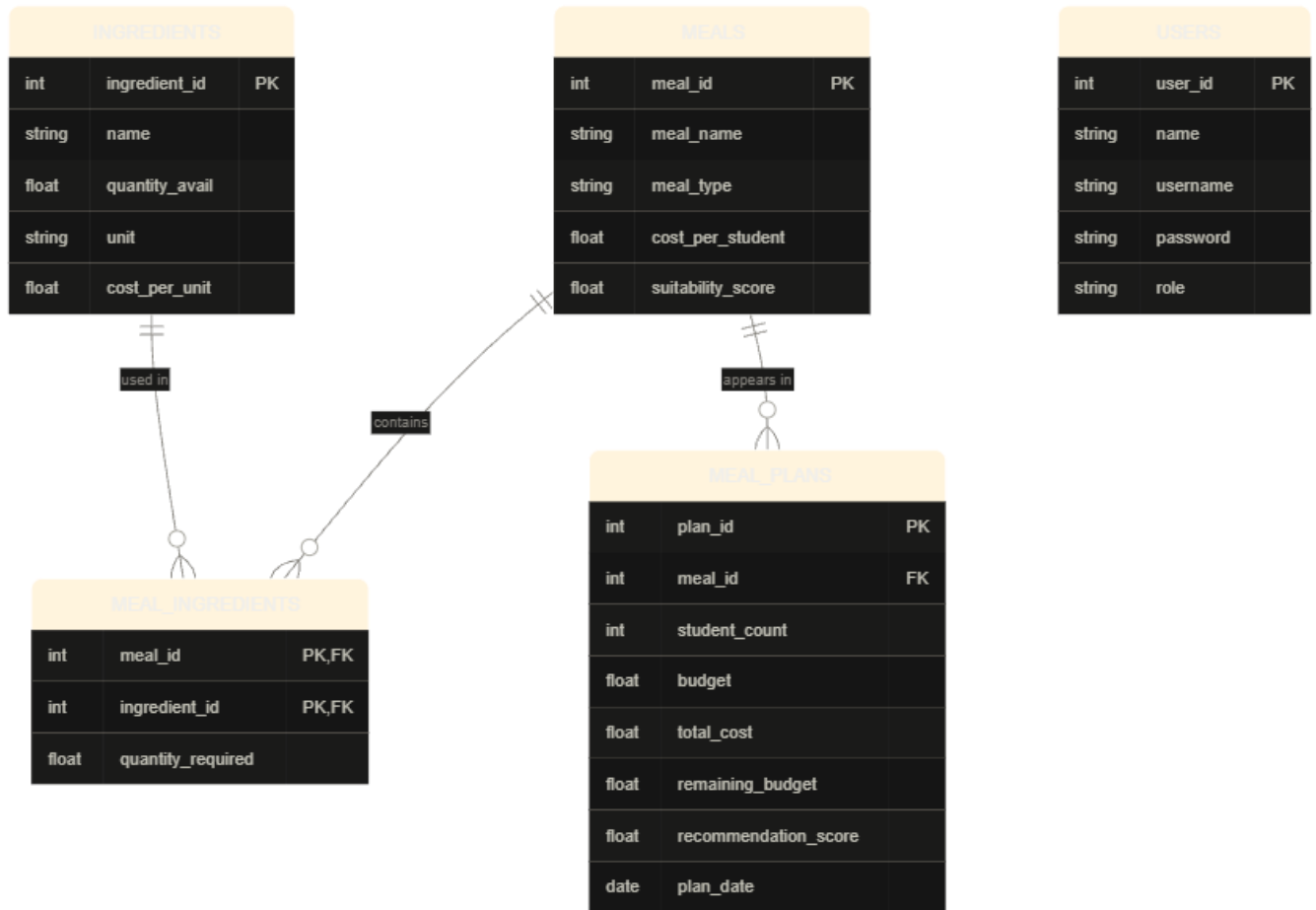
MEALS 1 ----- M MEAL_PLANS

In simple terms:

- One meal can have multiple ingredients.
- One ingredient can be used in multiple meals.
- meal_ingredients connects meals and ingredients.
- One meal can appear in multiple saved meal plans.

The meal_plans table stores the result of the recommendation process.

The ER Diagram shows how FoodWise stores its information in MySQL. Student records, ingredients, meals and generated meal plans are stored separately. The meal_ingredients table acts as a junction table between meals and ingredients because a meal can require several ingredients. The meal_plans table stores the generated recommendation and its financial details.



DESIGN DECISIONS AND RATIONALE

This section describes the technical decisions made when developing FoodWise.

8.1 Choice of Java

Java was chosen as the programming language since the project involves implementation of Java programming concepts in practice. FoodWise uses classes, objects, inheritance, abstraction, interfaces, collections, exception handling and file handling.

8.2 Choice of MySQL

MySQL was selected as the database to store the application data.

The system needs to store information about:

- Students
- Ingredients
- Meals
- Meal ingredients
- Saved meal plans
- Application users

Storing this information in a database facilitates its retrieval and updating whenever the application is run again.

8.3 Use of JDBC

JDBC is used to connect the Java application and MySQL.

The DAO classes use JDBC operations including:

- Connection
- PreparedStatement
- Statement
- ResultSet

This keeps the database related code in the DAO layer only and avoids mixing of SQL queries with the user interface.

8.4 Layered Structure

The packages below were created:

```
foodwise
|
|— model
|— dao
|— service
|— exception
|— util
|— main
```

The purpose of dividing the code into packages is to improve its readability and maintainability.

For example, the model classes represent data, DAO classes communicate with the database and service classes contain the main application logic.

8.5 Recommendation Approach

Instead of selecting a meal based only on its cost level, FoodWise applies a weighted recommendation score.

The current scoring involves three factors:

Factor	Weight
Budget	30%
Meal Suitability	40%
Resource Efficiency	30%

The factor of meal suitability has a higher weight since the main objective of the system is not only to find a low-cost meal but also to consider whether the meal is suitable for the students.

The system filters out meals that do not satisfy the budget or ingredient availability conditions before applying the recommendation strategy.

8.6 Ingredient Availability Check

Ingredient availability was added since a meal should not be recommended if its required ingredients are not available in sufficient quantity.

For example, if a meal requires a certain amount of rice per student, the system checks the available rice against the total amount required for the entered number of students.

This makes the recommendation more practical since it considers the actual stock stored in the database.

8.7 Use of Interface and Polymorphism

The recommendation logic uses an interface called:

RecommendationStrategy

The RecommendationEngine implements this interface.

This decision demonstrates the use of abstraction and polymorphism and also makes it easier to add another recommendation strategy in the future without changing the complete application structure.

8.8 Use of Prepared Statements

Prepared statements are used for database operations where user or application values are inserted into SQL queries.

This provides a cleaner way of passing values to SQL statements and is safer than directly constructing SQL queries using string concatenation.

8.9 Console-Based Interface

A menu-driven console interface was selected instead of developing a web or graphical interface.

The main reason was to maintain focus on Java programming, database connectivity and application logic. It also allowed more time to be spent on implementing the

actual recommendation and database modules.

The interface provides options such as:

1. Student Management
2. Ingredient Management
3. Meal Management
4. Generate Meal Recommendation
5. View Saved Meal Plans
6. Generate Report
7. Exit

8.10 File-Based Report Generation

FoodWise generates a basic text report containing information about the generated meal plan.

The report contains details such as:

- Recommended meal
- Number of students
- Budget
- Total cost
- Remaining budget
- Recommendation score
- Date

This provides an additional output from the system apart from the console display.

8.11 Overall Design Rationale

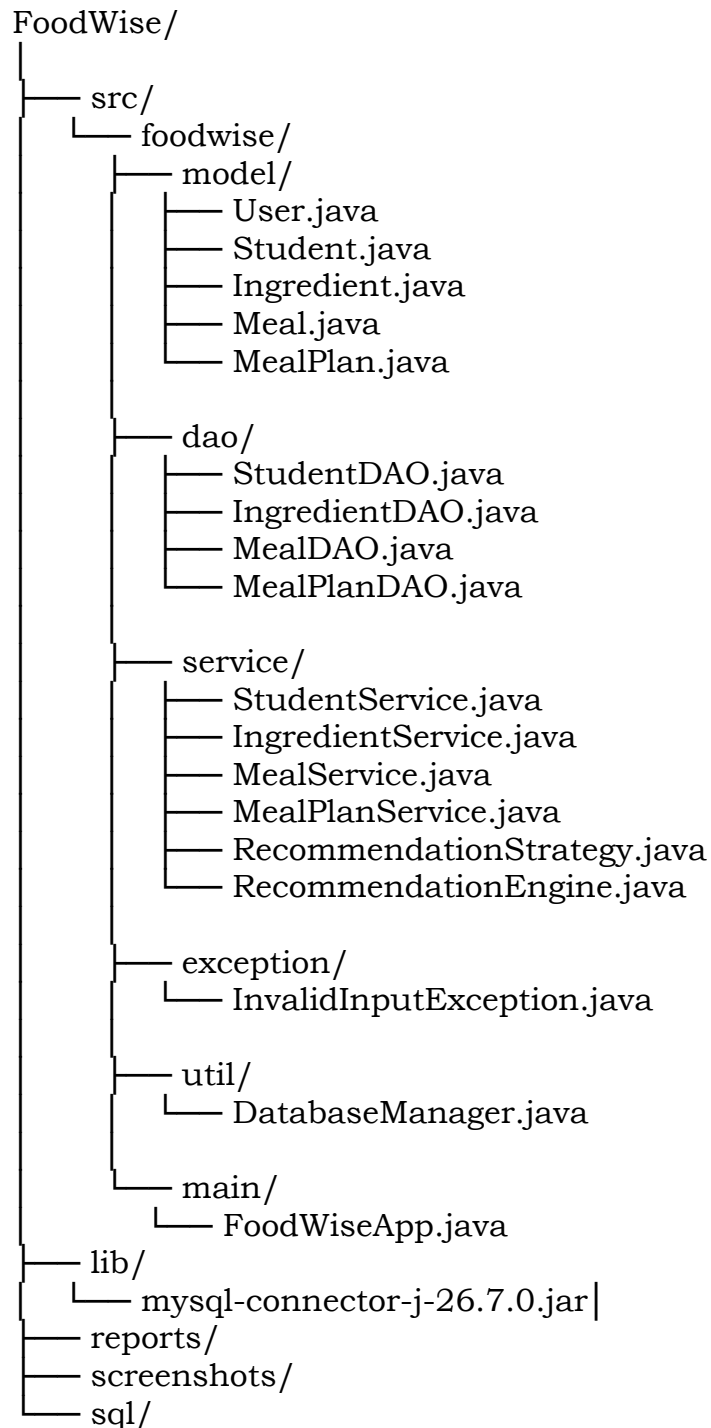
The overall design was kept simple and modular so that each part of the system has a clear responsibility. The project does not depend on external APIs or machine-learning frameworks. Instead, the recommendation logic is implemented using Java conditions, calculations and database information.

This approach keeps the project manageable while still demonstrating the Java and database concepts required for the course.

IMPLEMENTATION

The implementation of FoodWise was done using Java with JDBC and MySQL. The application is divided into different packages so that database operations, application logic and data classes are kept separately.

9.1 Project Structure



9.2 Model Layer

The model layer contains classes that represent the main objects used in the application.

The major model classes are:

- User
- Student
- Ingredient
- Meal
- MealPlan

These classes mainly contain data fields, constructors, getters, setters and required methods.

User and Student

User is an abstract class containing common user information such as:

userId

name

username

password

Student extends the User class and adds:

age

groupName

This demonstrates **inheritance** in Java.

The getRole() method is declared in the abstract User class and implemented in Student.

9.3 Ingredient Model

The Ingredient class represents the ingredients available for meal planning.

It contains information such as:

ingredientId

name

quantityAvailable

unit

costPerUnit

For example, the database can store ingredients such as rice, dal, vegetables, potato, wheat flour and milk.

The available quantity is important because the recommendation module checks whether enough ingredients are available for the selected number of students.

9.4 Meal Model

The Meal class represents the meal options available in the system.

The main attributes are:

mealId

mealName

mealType

costPerStudent

suitabilityScore

The costPerStudent is used when calculating the total cost of a meal, while suitabilityScore is one of the factors used by the recommendation engine.

9.5 MealPlan Model

The MealPlan class stores the result of the recommendation process.

It contains:

planId
mealId
studentCount
budget
totalCost
remainingBudget
recommendationScore
planDate

This object is created after a suitable meal has been selected and can then be saved into the database.

9.6 DAO Layer

DAO stands for **Data Access Object**.

The DAO layer is responsible for communication with the MySQL database. Instead of writing SQL queries directly inside the main application, database operations are separated into DAO classes.

The project contains:

StudentDAO
IngredientDAO
MealDAO
MealPlanDAO

```
PreparedStatement statement =  
    connection.prepareStatement(sql);
```

This separation makes the database code easier to locate and maintain.

9.7 StudentDAO

StudentDAO handles operations related to the students table.

The implemented operations include:

- Add student
- Get all students
- Update student
- Delete student

The class uses JDBC PreparedStatement for insert, update and delete operations.

For example, adding a student uses an SQL statement similar to:

```
INSERT INTO students  
(name, age, group_name)  
VALUES (?, ?, ?)
```

The values are supplied separately through the prepared statement.

9.8 IngredientDAO

IngredientDAO manages ingredient information.

The main operations include:

- Add ingredient
- View all ingredients
- Update ingredient quantity

The DAO retrieves the current quantity from the database so that the application can use the stored stock information during recommendation generation.

9.9 MealDAO

MealDAO manages meal information and also performs the ingredient availability check.

The availability check joins the meal_ingredients and ingredients tables.

The basic logic is:

Required quantity

=

Quantity required per student × Number of students

The available ingredient quantity is then compared with the required quantity.

If the available quantity is sufficient for all required ingredients, the meal can continue to the recommendation stage.

9.10 MealPlanDAO

MealPlanDAO is responsible for storing and retrieving generated meal plans.

It provides operations for:

- Saving a meal plan
- Viewing saved meal plans

When a plan is saved, information such as budget, total cost, remaining budget, recommendation score and date is inserted into the meal_plans table.

The saved plans are later retrieved using a SQL SELECT query.

9.11 Service Layer

The service layer contains the main application logic between the user interface and DAO layer.

The main service classes are:

StudentService

IngredientService

MealService

MealPlanService

RecommendationEngine

The service classes also perform input validation before sending data to the DAO layer.

This prevents invalid values from being directly inserted into the database.

9.12 StudentService

StudentService handles student-related operations.

Before adding or updating a student, it checks conditions such as:

- Student name should not be empty.
- Age should be within the accepted range.
- Group name should not be empty.

After validation, the service creates or updates the Student object and sends it to StudentDAO.

9.13 IngredientService

IngredientService validates ingredient-related input.

The validation includes:

- Ingredient name should not be empty.
- Quantity should not be negative.
- Unit should be provided.
- Cost per unit should not be negative.

After validation, the data is passed to IngredientDAO.

9.14 MealService

MealService handles meal creation and retrieval.

While adding a meal, it validates:

- Meal name
- Meal type
- Cost per student
- Suitability score

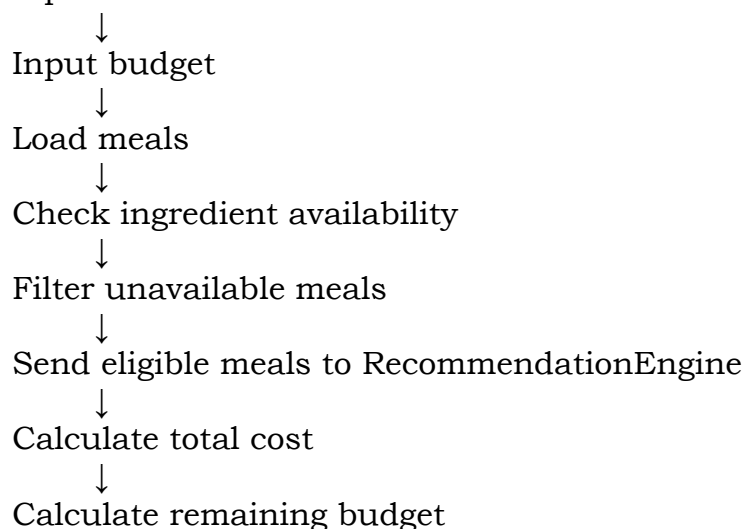
The suitability score is restricted to a value between **0 and 100**.

This prevents invalid suitability values from entering the recommendation system.

9.15 MealPlanService

MealPlanService controls the main recommendation workflow.

Input student count



↓
Create MealPlan

↓
Return result

The service also validates that the student count and budget are positive.

This class acts as an important connection between the user interface, database and recommendation logic.

9.16 RecommendationStrategy

FoodWise uses an interface named RecommendationStrategy.

Its main method is:

```
Meal recommendMeal(  
    List<Meal> meals,  
    int studentCount,  
    double budget  
);
```

The interface defines what a recommendation strategy should do without specifying all of its implementation details.

This demonstrates the use of **interfaces and polymorphism** in the project.

9.17 RecommendationEngine

The RecommendationEngine contains the main recommendation algorithm.

The system does not simply select the meal with the lowest cost. It considers three factors:

Budget Score = 30%

Suitability Score = 40%

Resource Efficiency = 30%

The final score is calculated by combining these factors.

Budget

The system checks how much of the available budget would be used.

Meals exceeding the available budget are not selected.

Suitability

The stored suitability score of the meal contributes 40% to the final recommendation score.

Resource Efficiency

The current implementation uses the meal's cost per student as the basis for this component.

The three scores are combined to produce the final recommendation score out of 100.

9.18 Recommendation Example

During testing, the system was given:

Number of students = 25

Budget = Rs. 1500

The system selected:

Meal = Vegetable Khichdi

Cost per student = Rs. 28
Total cost = Rs. 700
Remaining budget = Rs. 800
Suitability = 95
Recommendation = 93/100

The plan was generated after checking the available ingredient quantities.
This shows how the application uses both database information and Java-based calculation instead of selecting a meal randomly.

9.19 Exception Handling and Validation

FoodWise uses input validation and exception handling to avoid common runtime problems.

A custom exception class named:

`InvalidInputException`

is included for invalid application inputs.

The DAO classes also handle `SQLException` while communicating with MySQL.

For example, database errors are caught and an appropriate message is displayed rather than allowing the application to terminate unexpectedly.

9.20 JDBC Database Connectivity

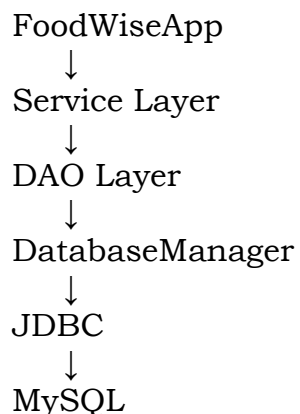
Database connectivity is handled through the `DatabaseManager` class.

The application uses a JDBC URL similar to:

`jdbc:mysql://localhost:3306/foodwise`

The `DatabaseManager` provides database connections to DAO classes.

The connection flow is:



This keeps the connection code in one place.

9.21 Database Setup and Configuration

FoodWise uses MySQL as the database and JDBC for connecting the Java application with the database. The database used by the application is named `foodwise`. The database contains tables for users, students, ingredients, meals, meal ingredients and meal plans.

A complete SQL setup file named `foodwise_setup.sql` is included in the project repository under the `sql` folder. This file can be used to create the database structure and insert the required sample data, including the 25-student dataset used during testing.

The application also includes an administrator login for accessing the FoodWise system. For demonstration purposes, the application login is:

Username: admin

Password: admin123

The MySQL username and password are configured locally because database credentials are specific to each computer. Personal MySQL credentials are not stored in the public GitHub repository.

9.22 File Handling and Report Generation

FoodWise also uses Java file handling to generate a text report.

After a recommendation is generated, the application can create a report containing the meal plan information.

The report includes details such as:

Meal Name

Number of Students

Budget

Total Cost

Remaining Budget

Recommendation Score

Plan Date

The reports are stored in the project's reports folder.

This also demonstrates the use of Java file I/O in a practical application.

9.23 Main Application

FoodWiseApp.java acts as the main entry point of the application.

It provides the menu-driven interface and connects the different modules.

The main menu contains:

1. Student Management
2. Ingredient Management
3. Meal Management
4. Generate Meal Recommendation
5. View Saved Meal Plans
6. Generate Report
7. Exit

The application takes input from the administrator, calls the required service method and displays the result.

9.23 Database Tables Used

The MySQL database contains six main tables:

Table	Purpose
users	Stores application login information

Table	Purpose
students	Stores student details
ingredients	Stores available ingredient stock
meals	Stores meal information
meal_ingredients	Connects meals with their required ingredients
meal_plans	Stores generated meal plans

The database structure allows the application to keep the information even after the program is closed.

9.24 Overall Implementation Flow

The complete implementation can be summarized as:

Administrator



FoodWiseApp



Service Layer



Validation



DAO Layer



JDBC / DatabaseManager



MySQL



Data returned to Service



Recommendation Processing



MealPlan



Console Output / Database / Report

The implementation follows the project requirement of using a modular structure with meaningful classes and proper separation of responsibilities.

SCREENSHOTS AND RESULTS

This section presents the actual working results of the FoodWise application. The screenshots show the main screens of the system and demonstrate that the implemented modules are working with the MySQL database.

The application was tested using different student counts and budget values. The recommendation module was also tested with the available ingredient quantities before generating the final meal plan.

10.1 Application Welcome Screen

The application starts with the FoodWise welcome screen. It displays the project name and tests the database connection before continuing.

The successful connection message confirms that the Java application is able to connect with the MySQL foodwise database using JDBC.

Expected output:

```
s\Anushka\AppData\Local\Temp\cp_em92hpsgs5huwflx6k0fxct6k.argfile' 'foodwise.main.FoodWiseApp'
=====
                WELCOME TO FOODWISE
=====
Java-Based Intelligent Meal Planning
and Resource Optimization System
=====

Database connection:SUCCESS!
```

Figure 10.1: FoodWise welcome screen and database connection

10.2 Administrator Login

After the database connection is established, the administrator login screen is displayed.

The administrator enters the application username and password. The login details are checked against the users table in the database.

After successful authentication, the main menu is displayed.

```
s\Anushka\AppData\Local\Temp\cp_em92hpsgs5huwflx6k0fxct6k.argfile' 'foodwise.main.FoodwiseApp'
=====
                WELCOME TO FOODWISE
=====
Java-Based Intelligent Meal Planning
and Resource Optimization System
=====

Database connection:SUCCESS!

-----
                ADMIN LOGIN
-----

Username: admin
Password: admin123

Login successful!
Welcome, Foodwise Administrator!
```

Figure 10.2: Administrator login

10.3 Main Menu

The main menu provides access to the different modules of the application.

1. Student Management
2. Ingredient Management
3. Meal Management
4. Generate Meal Recommendation
5. View Saved Meal Plans
6. Generate Report
7. Exit

```
Login successful!
Welcome, Foodwise Administrator!
```

```
=====
                FOODWISE MENU
=====
1. Student Management
2. Ingredient Management
3. Meal Management
4. Generate Meal Recommendation
5. View Saved Meal Plans
6. Generate Report
7. Exit
=====
Enter your choice: 
```

Figure 10.3: FoodWise main menu

10.4 Student Management

The Student Management module allows the administrator to work with student records.

The implemented operations include:

- Add student
- View students
- Update student
- Delete student

The student records are stored in the students table of the MySQL database.

```
=====
Enter your choice: 1

-----
                STUDENT MANAGEMENT
-----

1. View Students
2. Add Student
3. Update Student
4. Delete Student
5. Back
Enter choice: █
```

Figure 10.4: Student Management module

If we select 1.View Students, O/P observed is:

1. View Students
2. Add Student
3. Update Student
4. Delete Student
5. Back

Enter choice: 1

ID	NAME	AGE	GROUP
1	ANUSHKA SHARMA	18	Group A
2	KUSH SHARMA	19	Group A
3	PREETI CHOUHAN	18	Group B
4	KANIKA TIWARI	19	Group B
5	BADRIKANATH PRAHARAJ	20	Group C
6	SREERAM MV	19	Group C
7	ARJUN MEHTA	20	General
8	RIYA SINGH	19	General
9	ADITYA VERMA	21	General
10	NEHA GUPTA	20	General
11	ROHAN PATEL	22	General
12	SIMRAN KAUR	20	General
13	AMAN KHAN	21	General
14	PRIYA NAGAR	19	General
15	RAHUL YADAV	22	General
16	SAKSHI JAIN	20	General
17	VIVEK SHARMA	21	General
18	TANVI GUPTA	20	General
19	MOHIT RAJ	21	General
20	ISHA MALHOTRA	20	General
21	NIKHIL SINGH	22	General
22	AISHWARYA DAS	20	General
23	KARAN THAKUR	21	General
24	MEGHA SHAH	20	General
25	DEVENDRA KUMAR	22	General

10.5 Ingredient Management

The Ingredient Management module is used to maintain the available ingredients.

The system stores:

- Ingredient name
- Available quantity
- Unit
- Cost per unit

The quantity information is later used when checking whether sufficient ingredients are available for the required number of students.

```
=====
                        FOODWISE MENU
=====
1. Student Management
2. Ingredient Management
3. Meal Management
4. Generate Meal Recommendation
5. View Saved Meal Plans
6. Generate Report
7. Exit
=====
Enter your choice: 2

-----
                        INGREDIENT MANAGEMENT
-----
1. View Ingredients
2. Add Ingredient
3. Update Quantity
4. Back
Enter choice: 1
```

Figure 10.5: Ingredient Management module

If we enter option 1.View Ingredients, O/P observed is:

```
-----
                        INGREDIENT MANAGEMENT
-----
1. View Ingredients
2. Add Ingredient
3. Update Quantity
4. Back
Enter choice: 1

=====
=====
```

ID	NAME	QUANTITY	UNIT	COST/UNIT
1	Rice	50.00	kg	Rs.55.00
2	Dal	25.00	kg	Rs.110.00
3	Vegetables	30.00	kg	Rs.60.00
4	Potato	25.00	kg	Rs.35.00
5	Wheat Flour	30.00	kg	Rs.45.00
6	Milk	20.00	litre	Rs.60.00
7	Fruits	75.00	kg	Rs.110.00

```
=====
=====
```

10.6 Meal Management

The Meal Management module allows the administrator to view existing meals and add new meal options.

Each meal contains information such as:

- Meal name
- Meal type
- Cost per student
- Suitability score

The meal information is stored in the meals table.

```
=====
                        FOODWISE MENU
=====
1. Student Management
2. Ingredient Management
3. Meal Management
4. Generate Meal Recommendation
5. View Saved Meal Plans
6. Generate Report
7. Exit
=====
Enter your choice: 3

-----
                        MEAL MANAGEMENT
-----
1. View Meals
2. Add Meal
3. Back
Enter choice: █
```

Figure 10.6: Meal Management module

If we enter option 1.View Meals, O/P observed is:

```
-----
                        MEAL MANAGEMENT
-----
1. View Meals
2. Add Meal
3. Back
Enter choice: 1

=====
ID      MEAL                                TYPE      COST/STUDENT      SUITABILITY
=====
1       Dal Rice with Vegetables           Lunch     Rs.32.00           90
2       Vegetable Khichdi                   Lunch     Rs.28.00           95
3       Roti Potato Curry                     Lunch     Rs.30.00           85
```

10.7 Meal Recommendation Input and Result

The recommendation module asks the administrator to enter the required number of students and the available budget.

For example:

Enter number of students: 25

Enter available budget: Rs. 1500

The system then loads the available meals and checks their eligibility.

```
=====
                        FOODWISE MENU
=====
1. Student Management
2. Ingredient Management
3. Meal Management
4. Generate Meal Recommendation
5. View Saved Meal Plans
6. Generate Report
7. Exit
=====
Enter your choice: 4

=====
                        INTELLIGENT RECOMMENDATION
=====
Number of students: 25
Available budget (Rs.): 1500

=====
                        RECOMMENDATION RESULT
=====
Recommended Meal : Vegetable Khichdi
Meal Type       : Lunch
Students        : 25
Cost / Student  : Rs.28.00
Total Cost      : Rs.700.00
Budget          : Rs.1500.00
Remaining Budget : Rs.800.00
Suitability     : 95/100
Recommendation Score : 93.00/100
Plan Date       : 2026-09-18
=====
Save this meal plan? (yes/no): █
```

Figure 10.7: Meal recommendation

The recommendation result displays the selected meal and its calculated details. The result shows that the total calculated cost is within the given

10.8 Saving the Meal Plan

After a recommendation is generated, the administrator gets the option to save the meal plan.

When the administrator selects the save option, the plan is inserted into the meal_plans table.

The saved information includes:

- Selected meal
- Student count
- Budget
- Total cost
- Remaining budget
- Recommendation score
- Plan date

A successful save message is displayed after the database operation is completed.

```
=====
Save this meal plan? (yes/no): yes
Meal plan saved successfully!
```

Figure 10.8: Saving generated meal plan

10.9 View Saved Meal Plans

The View Saved Meal Plans option displays plans that have previously been stored in the database.

During testing, the application successfully displayed saved plans containing different student counts and budgets.

```
=====
Enter your choice: 5

=====
                SAVED MEAL PLANS
=====

-----
Plan ID       : 1
Meal          : Vegetable Khichdi
Students      : 25
Budget        : Rs.1500.00
Total Cost    : Rs.700.00
Remaining Budget : Rs.800.00
Score         : 93.00/100
Date          : 2026-09-18
-----
```

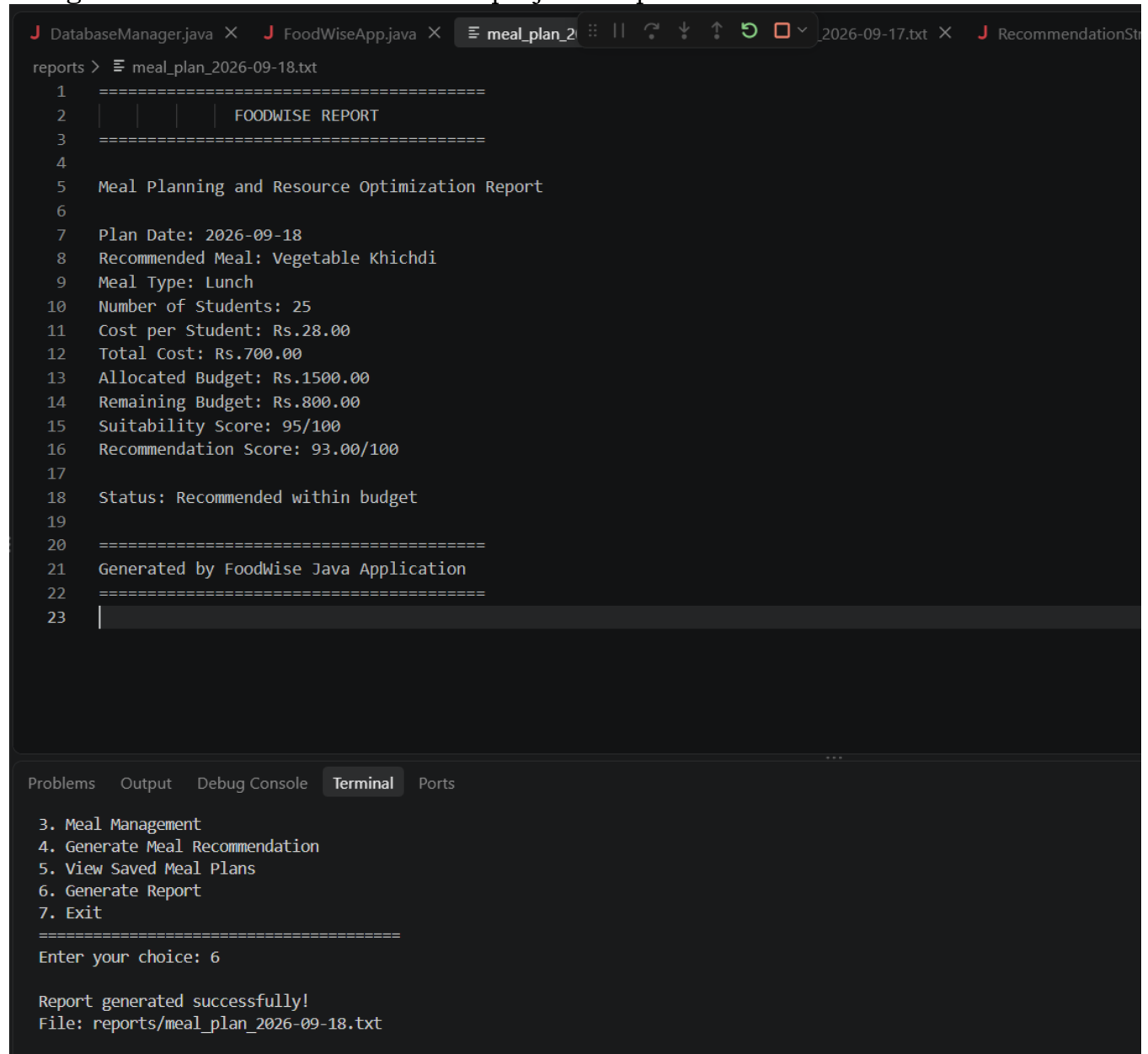
Figure 10.9: Saved meal plans

10.10 Generated Report

FoodWise also provides an option to generate a text report for the generated meal plan.

The report contains the main details of the recommendation, including the meal name, student count, budget, total cost, remaining budget, recommendation score and plan date.

The generated files are stored in the project's reports folder.



The screenshot shows an IDE with two windows. The top window, titled 'meal_plan_2', displays the content of a text file 'meal_plan_2026-09-18.txt'. The file contains a formatted report for a meal plan. The bottom window, titled 'Terminal', shows the application's command-line interface where the user has selected option 6 to generate a report, and the application has confirmed the report was generated successfully at the specified file path.

```

DatabaseManager.java X FoodWiseApp.java X meal_plan_2 || ? ↓ ↑ ↺ □ v _2026-09-17.txt X RecommendationSt
reports > meal_plan_2026-09-18.txt
1  =====
2  | | | | FOODWISE REPORT
3  =====
4
5  Meal Planning and Resource Optimization Report
6
7  Plan Date: 2026-09-18
8  Recommended Meal: Vegetable Khichdi
9  Meal Type: Lunch
10 Number of Students: 25
11 Cost per Student: Rs.28.00
12 Total Cost: Rs.700.00
13 Allocated Budget: Rs.1500.00
14 Remaining Budget: Rs.800.00
15 Suitability Score: 95/100
16 Recommendation Score: 93.00/100
17
18 Status: Recommended within budget
19
20 =====
21 Generated by FoodWise Java Application
22 =====
23 |

```

```

Problems Output Debug Console Terminal Ports
3. Meal Management
4. Generate Meal Recommendation
5. View Saved Meal Plans
6. Generate Report
7. Exit
=====
Enter your choice: 6

Report generated successfully!
File: reports/meal_plan_2026-09-18.txt

```

Figure 10.10: Generated meal plan report

10.11 Test Case Results

Different input values were used to check the recommendation module.

```
=====
                    INTELLIGENT RECOMMENDATION
=====
Number of students: 6
Available budget (Rs.): 200

=====
                    RECOMMENDATION RESULT
=====
Recommended Meal : Vegetable Khichdi
Meal Type       : Lunch
Students        : 6
Cost / Student  : Rs.28.00
Total Cost      : Rs.168.00
Budget          : Rs.200.00
Remaining Budget : Rs.32.00
Suitability     : 95/100
Recommendation Score : 83.00/100
Plan Date       : 2026-09-18
=====
Save this meal plan? (yes/no): no
```

```
=====
                    INTELLIGENT RECOMMENDATION
=====
Number of students: 25
Available budget (Rs.): 1000

=====
                    RECOMMENDATION RESULT
=====
Recommended Meal : Vegetable Khichdi
Meal Type       : Lunch
Students        : 25
Cost / Student  : Rs.28.00
Total Cost      : Rs.700.00
Budget          : Rs.1000.00
Remaining Budget : Rs.300.00
Suitability     : 95/100
Recommendation Score : 88.00/100
Plan Date       : 2026-09-18
=====
Save this meal plan? (yes/no): no
```

```
=====
INTELLIGENT RECOMMENDATION
=====
Number of students: 5
Available budget (Rs.): 800

=====
RECOMMENDATION RESULT
=====
Recommended Meal : Vegetable Khichdi
Meal Type       : Lunch
Students        : 5
Cost / Student  : Rs.28.00
Total Cost      : Rs.140.00
Budget          : Rs.800.00
Remaining Budget : Rs.660.00
Suitability     : 95/100
Recommendation Score : 93.00/100
Plan Date       : 2026-09-18

=====
Save this meal plan? (yes/no): no
```

```
=====
INTELLIGENT RECOMMENDATION
=====
Number of students: 50
Available budget (Rs.): 500

Recommendation error: No suitable meal found within the given budget.
```

10.12 Exit

```
FOODWISE MENU
=====
1. Student Management
2. Ingredient Management
3. Meal Management
4. Generate Meal Recommendation
5. View Saved Meal Plans
6. Generate Report
7. Exit
=====
Enter your choice: 7

Thank you for using Foodwise!
```

10.13 Overall Result

FoodWise successfully provides a small Java-based solution for meal planning and resource management. The final application connects Java with MySQL using JDBC and combines database information with Java-based recommendation logic.

The project demonstrates that the selected Java concepts can be used together in a practical application. The system is also structured so that additional meal options, ingredients and recommendation rules can be added later.

The implementation produced the expected results during testing.

TESTING

Testing was performed to check whether the FoodWise application was working according to its requirements. The main purpose of testing was to identify errors in database connectivity, input validation, student management, meal recommendation and report generation.

The application was tested using different student counts, budget values and meal-planning conditions.

11.1 Testing Approach

FoodWise was tested mainly through manual functional testing. Inputs were entered through the console interface and the displayed results were compared with the expected output.

The following areas were tested:

- Database connection
- Administrator login
- Student management
- Ingredient management
- Meal management
- Meal recommendation
- Ingredient availability
- Saving and retrieving meal plans
- Report generation
- Invalid input handling

11.2 Functional Test Cases

Test ID	Module	Test Input/Action	Expected Result	Observed Result	Status
T01	Database Connection	Start the application	Database connection should be established	Connection was successful	Pass
T02	Admin Login	Enter valid username and password	Main menu should be displayed	Main menu was displayed	Pass
T03	Admin Login	Enter invalid login details	Error message should be displayed	Invalid login is rejected	Pass
T04	Student Management	Add a student with valid details	Student should be inserted into the database	Student record was added	Pass
T05	Student Management	View student records	Stored students should be displayed	Student records were displayed	Pass
T06	Student Management	Update student information	Selected student details should be updated	Update operation was supported	Pass
T07	Student Management	Delete a student	Selected student should be removed	Delete operation was supported	Pass
T08	Ingredient Management	Add ingredient with valid quantity	Ingredient should be stored	Ingredient was added	Pass
T09	Ingredient Management	Update ingredient quantity	Available quantity should change	Quantity update was supported	Pass
T10	Meal Management	Add a meal with valid details	Meal should be stored	Meal was added	Pass
T11	Recommendation	Enter 5 students and Rs. 800 budget	A suitable meal should be recommended	Vegetable Khichdi was recommended	Pass
T12	Recommendation	Enter 25 students and Rs. 1500 budget	Total cost should remain within budget	Vegetable Khichdi was recommended	Pass
T13	Ingredient Check	Request a meal for a large number of students	Insufficient ingredients should exclude the meal	Availability check was applied	Pass
T14	Meal Plan	Save a generated recommendation	Plan should be saved in MySQL	Plan was saved successfully	Pass
T15	Saved Plans	View previously saved plans	Stored plans should be displayed	Saved plans were retrieved	Pass
T16	Report Generation	Generate report after recommendation	Text report should be created	Report was generated	Pass



```
-----
                        STUDENT MANAGEMENT
-----

1. View Students
2. Add Student
3. Update Student
4. Delete Student
5. Back
Enter choice: 2

--- ADD STUDENT ---
Student name: Shreya Singh
Age: 20
Group name: General
Student added successfully!

-----
                        STUDENT MANAGEMENT
-----

1. View Students
2. Add Student
3. Update Student
4. Delete Student
5. Back
Enter choice: 1

=====
ID      NAME                                     AGE      GROUP
=====
1       ANUSHKA SHARMA                             18      Group A
2       KUSH SHARMA                                   19      Group A
3       PREETI CHOUHAN                               18      Group B
4       KANIKA TIWARI                                 19      Group B
5       BADRIKANATH PRAHARAJ                         20      Group C
6       SREERAM MV                                    19      Group C
7       ARJUN MEHTA                                   20      General
8       RIYA SINGH                                    19      General
9       ADITYA VERMA                                  21      General
10      NEHA GUPTA                                    20      General
11      ROHAN PATEL                                   22      General
12      SIMRAN KAUR                                  20      General
13      AMAN KHAN                                    21      General
14      PRIYA NAGAR                                  19      General
15      RAHUL YADAV                                  22      General
16      SAKSHI JAIN                                   20      General
17      VIVEK SHARMA                                  21      General
18      TANVI GUPTA                                  20      General
19      MOHIT RAJ                                    21      General
20      ISHA MALHOTRA                                20      General
21      NIKHIL SINGH                                 22      General
22      AISHWARYA DAS                                20      General
23      KARAN THAKUR                                  21      General
24      MEGHA SHAH                                   20      General
25      DEVENDRA KUMAR                              22      General
140    Shreya Singh                                20      General
=====
```



1. View Students
2. Add Student
3. Update Student
4. Delete Student
5. Back

Enter choice: 4

ID	NAME	AGE	GROUP
1	ANUSHKA SHARMA	18	Group A
2	KUSH SHARMA	19	Group A
3	PREETI CHOUHAN	18	Group B
4	KANIKA TIWARI	19	Group B
5	BADRIKANATH PRAHARAJ	20	Group C
6	SREERAM MV	19	Group C
7	ARJUN MEHTA	20	General
8	RIYA SINGH	19	General
9	ADITYA VERMA	21	General
10	NEHA GUPTA	20	General
11	ROHAN PATEL	22	General
12	SIMRAN KAUR	20	General
13	AMAN KHAN	21	General
14	PRIYA NAGAR	19	General
15	RAHUL YADAV	22	General
16	SAKSHI JAIN	20	General
17	VIVEK SHARMA	21	General
18	TANVI GUPTA	20	General
19	MOHIT RAJ	21	General
20	ISHA MALHOTRA	20	General
21	NIKHIL SINGH	22	General
22	AISHWARYA DAS	20	General
23	KARAN THAKUR	21	General
24	MEGHA SHAH	20	General
25	DEVENDRA KUMAR	22	General
140	Shreya Singh	20	General

Enter student ID to delete: 22

Student deleted successfully!



Enter your choice: 2

INGREDIENT MANAGEMENT

1. View Ingredients
2. Add Ingredient
3. Update Quantity
4. Back

Enter choice: 2

--- ADD INGREDIENT ---

Ingredient name: Eggs

Available quantity: 250

Unit (kg/litre/etc.): dozen

Cost per unit: 11

Ingredient added successfully!

INGREDIENT MANAGEMENT

1. View Ingredients
2. Add Ingredient
3. Update Quantity
4. Back

Enter choice: 1

ID	NAME	QUANTITY	UNIT	COST/UNIT
1	Rice	50.00	kg	Rs.55.00
2	Dal	25.00	kg	Rs.110.00
3	Vegetables	30.00	kg	Rs.60.00
4	Potato	25.00	kg	Rs.35.00
5	Wheat Flour	30.00	kg	Rs.45.00
6	Milk	20.00	litre	Rs.60.00
7	Fruits	75.00	kg	Rs.110.00
8	Eggs	250.00	dozen	Rs.11.00

Enter your choice: 3

MEAL MANAGEMENT

1. View Meals
2. Add Meal
3. Back

Enter choice: 2

--- ADD MEAL ---

Meal name: Curd Rice

Meal type: Breakfast

Cost per student: 15

Suitability score (0-100): 95

Meal added successfully!

MEAL MANAGEMENT

1. View Meals
2. Add Meal
3. Back

Enter choice: 1

ID	MEAL	TYPE	COST/STUDENT	SUITABILITY
1	Dal Rice with Vegetables	Lunch	Rs.32.00	90
2	Vegetable Khichdi	Lunch	Rs.28.00	95
3	Roti Potato Curry	Lunch	Rs.30.00	85
4	Curd Rice	Breakfast	Rs.15.00	95

11.3 Recommendation Test Cases

The recommendation module was tested using different numbers of students and budget values.

Test ID	Number of Students	Budget	Recommended Meal	Total Cost	Remaining Budget	Status
R01	5	Rs. 800	Vegetable Khichdi	Rs. 140	Rs. 660	Pass
R02	6	Rs. 200	Vegetable Khichdi	Rs. 168	Rs. 32	Pass
R03	25	Rs. 1500	Vegetable Khichdi	Rs. 700	Rs. 800	Pass
R04	50	Rs. 500	No suitable meal found within the given budget.	-	-	Pass

The results show that the total meal cost changes according to the number of students. The application also calculates the remaining budget automatically.

11.4 Input Validation Test Cases

The service layer contains validation rules to prevent invalid data from being processed.

Test ID	Invalid Input	Expected Behaviour
V01	Empty student name	Display an invalid-name message
V02	Student age below the accepted range	Reject the student record
V03	Empty group name	Reject the input
V04	Empty ingredient name	Reject the ingredient
V05	Negative ingredient quantity	Display a validation error
V06	Negative ingredient cost	Reject the input
V07	Empty meal name	Reject the meal
V08	Meal cost equal to or below zero	Display a validation error
V09	Suitability score below 0 or above 100	Reject the score
V10	Student count equal to or below zero	Reject the recommendation request
V11	Budget equal to or below zero	Reject the recommendation request
V12	Empty meal type	Reject the meal input

These checks help reduce incorrect data entry and prevent invalid values from reaching the database.

Enter choice: 3

```
=====
```

ID	NAME	AGE	GROUP
=====			
1	ANUSHKA SHARMA	18	Group A
2	KUSH SHARMA	19	Group A
3	PREETI CHOUHAN	18	Group B
4	KANIKA TIWARI	19	Group B
5	BADRIKANATH PRAHARAJ	20	Group C
6	SREERAM MV	19	Group C
7	ARJUN MEHTA	20	General
8	RIYA SINGH	19	General
9	ADITYA VERMA	21	General
10	NEHA GUPTA	20	General
11	ROHAN PATEL	22	General
12	SIMRAN KAUR	20	General
13	AMAN KHAN	21	General
14	PRIYA NAGAR	19	General
15	RAHUL YADAV	22	General
16	SAKSHI JAIN	20	General
17	VIVEK SHARMA	21	General
18	TANVI GUPTA	20	General
19	MOHIT RAJ	21	General
20	ISHA MALHOTRA	20	General
21	NIKHIL SINGH	22	General
23	KARAN THAKUR	21	General
24	MEGHA SHAH	20	General
25	DEVENDRA KUMAR	22	General
140	Shreya Singh	20	General

Enter student ID to update: 450

New name: Khushi

New age: 18

New group: B

Student ID not found.

Similarly, certain testing cases were done.....

11.5 Testing of Report Generation

The report-generation feature was tested after creating a meal recommendation. The generated report was checked for the presence of:

- Meal name
- Number of students
- Budget
- Total cost
- Remaining budget
- Recommendation score
- Plan date

The report was successfully created as a text file in the reports folder.

11.6 Testing Summary

The main functional modules of the FoodWise application were tested through the console interface. The database connection, login system, student management system, ingredient management system, meal management system and the recommendation process were all tested and found to be working correctly.

The recommendation module was able to successfully calculate the total and remaining budget for different student numbers. The application was also able to check ingredient availability before selecting a meal and let the user save and view their plans.

Some additional testing could be done on the application in the future with larger sets of data and more options for meals.

11.7 Testing Limitations

The testing of the project was mainly done in a local development environment with a smaller set of meals and ingredients. The application was not tested with a database at production level and with multiple users active at the same time.

The current scope of testing was done with the focus on the main features needed for the academic project. A more extensive set of performance and security oriented tests should be done before deploying the database system in a real educational institution.

CHALLENGES FACED

During the development of FoodWise, several aspects of connecting the Java application with MySQL and integrating the various modules together proved challenging. These challenges provided valuable insight into how the concepts covered in the Java training could be used in practice.

12.1 Setting Up Java Environment

The Java development environment had to be configured correctly in VS Code, including the JDK and Java extensions, in order to compile and run the project.

This provided insights into what steps are needed to set up a Java application for development and execution.

12.2 JDBC and MySQL Connection

Connecting the Java application to MySQL proved a big part of the project.

The application makes use of JDBC in the form of the DatabaseManager class to access MySQL. The development process included debugging connection errors and reviewing JDBC settings.

This gave valuable insight into how Java communicates with an external database.

12.3 Maintaining Correct Database Column Names

When developing the DAO classes, it became evident that the SQL column names needed to match the MySQL table structure accurately.

For instance, the application makes use of field names such as:

cost_per_student

quantity_available

quantity_required

A discrepancy between the Java query and database column name resulted in SQL errors during testing.

This demonstrated the importance of keeping the Java code and database structure synchronized.

12.4 Ingredient Availability Logic

At first the recommendation engine focused purely on the budget and suitability. The system was later extended to take the availability of ingredients for the selected number of students into account.

For each meal, the required quantity is calculated using:

Required Quantity= Quantity Required Per Student × Number of Students

The result is compared to the available quantity in the database.

This enhanced the recommendation engine to be more realistic.

12.5 Input Validation

One of the most challenging aspects of the project was dealing with invalid inputs.

For instance, the system should not accept:

Negative ingredient quantities

Invalid suitability scores

Empty meal names

Invalid student ages

Zero or negative budgets

Zero or negative student counts

As a result, these validations were added to the service layer before sending data to the database.

12.6 Managing Different Modules

FoodWise comprises several modules including student, ingredient, meal, meal recommendation and report generation. Keeping these modules closely integrated together while avoiding responsibility overlap proved challenging.

The use of separate model, DAO, service, exception and utility packages helped with this issue.

LEARNING & KEY TAKEAWAYS

13.1 Practical Use of OOP

The project provided the opportunity to apply OOP concepts in practice.

Concepts such as:

Encapsulation

Inheritance

Abstraction

Interfaces

Polymorphism

were employed in various parts of the FoodWise application. For instance, Student extends the abstract User class while RecommendationEngine implements RecommendationStrategy.

13.2 Understanding JDBC

The concepts of JDBC were well demonstrated through this project:

```
Java Application
↓
DatabaseManager
↓
JDBC Connection
↓
PreparedStatement / Statement
↓
MySQL
↓
ResultSet
```

This provided an insight into how the database-driven Java applications work.

13.3 Importance of Modular Programming

The use of modular programming, i.e. separating the project into several packages, proved critical in making the project manageable.

For instance:

- Model classes represent data.
- DAO classes handle database operations.
- Service classes handle application logic.
- Utility classes handle common functionality.
- Exception classes handle invalid input.
- This separation facilitated easier debugging and modification of the project.

13.4 Applying Collections

Java collections were used to store and process multiple student, ingredient and meal objects.

This provided the opportunity to apply what was learned about working with List and ArrayList objects.

13.5 Understanding Decision-Making Logic

The recommendation module was a great way to apply the Java decision making logic.

While the initial approach was to pick a meal at random, FoodWise takes:

Budget
+
Suitability
+
Resource Efficiency
↓
Recommendation Score

into account when recommending a meal. This provided a practical application of Java condition checking, calculation and collections.

13.6 File Handling

The report generation feature helped with understanding Java file handling concepts. The meal plan that is generated can be written to a text file and saved in the project's report folder.

Overall Learning

The highlight of this project was that application development involves more than just writing Java classes. It was necessary to combine various concepts for successful application development.

FoodWise provided the hands-on experience that was needed when working with:

- ✓ Java programming
- ✓ Object-oriented design
- ✓ JDBC
- ✓ MySQL
- ✓ SQL queries
- ✓ Input validation
- ✓ Exception handling
- ✓ Collections
- ✓ File handling
- ✓ Modular application design
- ✓ Testing and debugging

The project was also great for understanding how a small practical scenario could be converted into a Java application.

FUTURE ENHANCEMENTS

Although FoodWise currently provides the main features required for student meal planning, the system can be extended further in the future. The current version focuses on Java, JDBC, MySQL and a rule-based recommendation approach. The following improvements can be considered in later versions.

14.1 Improved Recommendation System

The recommendation logic can be extended to consider more factors such as:

- Nutritional value
- Student age groups
- Dietary preferences
- Meal variety
- Ingredient wastage
- Seasonal ingredient availability

This could make the recommendations more detailed while keeping the existing budget and ingredient checks.

14.2 More Meal Options

More meals and meal types can be added to the database.

For example:

-  Breakfast
-  Lunch
-  Snacks
-  Dinner

Each meal could have its own ingredient requirements, cost and suitability score.

This would give the recommendation engine a larger set of choices.

14.3 Advanced Student Categories

The current system stores a group/category for students. In the future, separate categories could be created for different student requirements.

For example:

- ✚ General
- ✚ Hostel Students
- ✚ Day Scholars
- ✚ Age Groups
- ✚ Special Dietary Requirements

The recommendation system could then use these categories while selecting meals.

14.4 Improved Inventory Management

The ingredient module can be extended into a more complete inventory system.

Future versions could include:

- ✚ Automatic stock deduction after a meal plan is finalized
- ✚ Low-stock warnings
- ✚ Stock history
- ✚ Ingredient expiry dates
- ✚ Automatic calculation of required purchases

This could help reduce unnecessary wastage and make resource management more practical.

14.5 Graphical User Interface

The current application uses a console-based interface. A future version could provide a graphical interface using a Java GUI framework.

A GUI could provide:

- ✚ Forms for entering student details
- ✚ Tables for viewing records
- ✚ Buttons for managing meals and ingredients
- ✚ Better presentation of recommendation results
- ✚ Graphs for budget and resource analysis

14.6 Better Reporting and Analytics

The current system generates a basic text report. This can be extended to provide more detailed reports.

Future reports could include:

- Monthly meal expenditure
- Ingredient usage
- Budget utilization
- Number of meal plans generated
- Most frequently selected meals
- Stock consumption
- Charts and graphical summaries could also be added.

14.7 User Roles

The current application focuses mainly on administrator access. A future version could support different roles, such as:

- Administrator
- Meal Planner
- Inventory Manager
- Viewer
- Different users could then be given access to different parts of the system.

14.8 Improved Security

For a real-world deployment, the login system could be improved by:

- ❖ Hashing passwords
- ❖ Using environment variables for database credentials
- ❖ Applying stronger authentication
- ❖ Adding account management
- ❖ Maintaining login activity records

The current project is intended as an academic application and therefore uses a simpler local authentication approach.

14.9 Web or Mobile Version

FoodWise could eventually be converted into a web or mobile-based application so that meal planners could access it from different devices.

The existing Java and database logic could serve as a starting point for such an extension.

14.10 Conclusion of Future Enhancements

The current version of FoodWise provides the basic foundation for meal planning, ingredient management and recommendation generation. The system can be expanded with improved recommendation factors, inventory management, analytics, stronger security and a graphical or web-based interface.

These enhancements are not part of the current implementation but could be considered if the project is developed further.

CONCLUSION

FoodWise was developed as a Java platform for student meal planning and organization. The primary objective of the project was to integrate student information, meal, ingredient availability, and budgetary needs into a single system.

The application is designed to offer various modules for student, ingredients, and meal management while considering ingredient availability to recommend meals while also adhering to the budget constraints and suitability. This project aimed to develop an application through which student information, ingredient availability, and other needs could be integrated into a single platform.

This project helped apply several concepts learned in this course. Some of these concepts include object-oriented programming, inheritance, abstraction, interfaces, polymorphism, generic types, collections, exception-handling mechanisms, JDBC, and file I/O. This project involved the use of Java as the primary programming language, MySQL as the database management system, and JDBC driver as the middleware.

Several issues that included database connection problems, SQL column mismatch, input validation, and recommendation problems were resolved. Additionally, the project provided firsthand experience with several aspects of a Java application, including the use of JDBC drivers and how a Java application interacts with databases.

The current project version offers a firm foundation for future additions and improvements, including advanced nutritional recommendation systems, graphical user interface, inventory management, and advanced reporting features. In conclusion, the project offered a foundation for the development of a full-scale Java application for student meal planning and management while providing insight into the development process, ranging from database design to implementation and report generation.

REFERENCES

1. **Oracle Java Documentation** – Java programming concepts, classes, inheritance, interfaces, exception handling and collections.
2. **MySQL Documentation** – MySQL database concepts, SQL statements, tables, relationships and database management.
3. **MySQL Connector/J Documentation** – JDBC connectivity between Java applications and MySQL databases.
4. **VITyarthi Project Guidelines** – Project requirements, functional and non-functional requirements, design documentation, implementation, testing and report guidelines.
5. **CSE2006 – Programming in Java Course Material** – Java fundamentals, object-oriented programming, exception handling, collections, file handling and JDBC concepts.
6. **FoodWise Project Source Code and Database Schema** – Developed as part of this project for implementing student management, ingredient management, meal management, recommendation and meal-plan reporting.